

ETW3420 Assignment 3

Due Date: 13th November 2025

Student ID: 33202796

Student Name: Sim Hui Ling

Section 1: Data Collection and Preprocessing

Part A: Collect and merge data

```
gold_demand <- read_xlsx("Gold_demand.xlsx",
                        sheet = "Consumer", skip = 4)
gold_demand <- gold_demand[, -c(2:20)]
gold_demand <- gold_demand %>%
  rename(Country = ...1)

golddemand_us <- gold_demand %>%
  filter(Country == "United States") %>%
  pivot_longer(
    cols = starts_with("Q"),
    names_to = "Quarter",
    values_to = "Gold_Demand_Tonnes"
  ) %>%
  mutate(
    Quarter = gsub("", " 20", Quarter),
    Quarter = as.yearqtr(Quarter, format = "Q%q %Y")
  ) %>%
  select(Quarter, Gold_Demand_Tonnes)
```

Figure 1.1: Importing gold demand data from World Gold Council (2025)

Description: The first line of code for `gold_demand` shows the import of the dataset from the source, removed columns 2-20 and renamed a column initially named '`...1`' to '`Country`' to make analysis easier and cleaner. The second chunk of code, `golddemand_us`, is more catered towards the analysis as it is focused on the economy, market, demand of gold in the United States in a quarterly manner. Secondly, it also standardises the time/year dataset into quarters in the format of YYYY Q1. The cleaned dataset would only have the quarter and the values for each quarter.

```
gold_price <- read_xlsx("Gold_prices.xlsx",
                      sheet = "Quarterly_Avg", skip = 3)
gold_price <- gold_price %>%
  rename(Quarter = ...1) %>%
  slice(-(1:125)) %>%
  mutate(Quarter = as.yearqtr(as.Date(Quarter))) %>%
  select(Quarter, USD)
```

Figure 1.2: Importing gold price data from World Gold Council (2019)

Description: The first line of code for `gold_price` shows the importing of the dataset, which chooses the individual sheet from the entire dataset for the quarterly values. The second chunk of code for `gold_price` is accustomed to the aim of the analysis. The column '`...1`' which contained values for the quarters was renamed to `quarters`, removing rows 1-125 as for the analysis, it is focused on 2010 onwards. Lastly to standardise the dataset into the quarter format, which will make it easier for data merging later on.

```

us_inflation <- read_xls("us_inflation.xls", skip = 2)
us_inflation <- us_inflation %>%
  rename(Country = `Country Name`) %>%
  filter(Country == "United States") %>%
  pivot_longer(
    cols = `1960`:`2024`,
    names_to = "Year",
    values_to = "Inflation_Rate"
  ) %>%
  mutate(Year = as.numeric(Year)) %>%
  filter(Year >= 2010) %>%
  select(-c(2,3,4))

us_inflation <- us_inflation %>%
  mutate(Quarter = as.yearqtr(Year)) %>%
  complete(Quarter = seq(min(Quarter), max(Quarter), by = 0.25)) %>%
  fill(Inflation_Rate, .direction = "down") %>%
  select(-c(2,3))

```

Figure 1.4: Importing inflation (CPI) data from World Bank (2024)

Description: The first line of code shows the importation of the dataset. The second chunk of the code is aimed at cleaning the dataset for easier merging. Considering that the analysis is focused on quarterly data and the original dataset for inflation is yearly. Therefore, the third chunk of code aims at dividing each year's inflation rate into 0.25 for each quarter.

```

interest_rates <- read.csv("interest_rates.csv")
interest_rates <- interest_rates %>%
  rename(Date = observation_date) %>%
  slice(-(1:666)) %>%
  mutate(Date = mdy(Date)) %>%
  filter(month(Date) %in% c(3,6,9,12)) %>%
  mutate(Quarter = as.yearqtr(Date)) %>%
  select(-c(1)) %>%
  rename(interest_rate = FEDFUNDS)

```

Figure 1.5: Importing interest rate data from Federal Reserve Bank of St. Louis (2025)

Description: The first line of code shows importing the dataset. The second chunk of code does further cleaning of removing unnecessary rows, filter the months for March, June, September and December for the quarters as this is a monthly dataset. Lastly, to standardize the dates, its put into the quarter format of YYYY Q1.

```

usd_index <- read_csv("usd_index.csv", show_col_types = FALSE)
usd_index <- usd_index %>%
  mutate(Date = as.Date(observation_date, format = "%m/%d/%y")) %>%
  filter(Date >= as.Date("2010-01-01")) %>%
  mutate(Quarter = as.yearqtr(Date)) %>%
  group_by(Quarter) %>%
  summarise(USD_Index = mean(DEXUSEU, na.rm = TRUE)) %>%
  arrange(Quarter)

```

Figure 1.6: Importing USD index (USD/EUR) data from Board of Governors of the Federal Reserve System (US), (2025)

Description: The first line of code shows the import of the dataset. The second chunk of code mainly focuses on the formatting of the date. The initial format of the date is m/d/yy. Therefore, I had first changed the format to a proper format of YYYY-MM-DD and the filter for only 2010 onwards. Followed by arranging and formatting the dates into quarters of YYYY Q1.

Part B: Data Cleaning

```

> sum(is.na(golddemand_us))
[1] 0
> sum(is.na(gold_price))
[1] 0
> sum(is.na(us_inflation))
[1] 0
> sum(is.na(usd_index))
[1] 0
> sum(is.na(interest_rates))
[1] 0

```

Figure 1.7: Checking for NA values

Description: To ensure the dataset is cleaned for further analysis and transformation, checking for missing values is crucial. After checking individual variables, there are no missing values.

Code	Output
<code>tsoutliers(golddemand_us\$Gold_Demand_Tonnes)</code>	<pre>\$index integer(0) \$replacements numeric(0)</pre>
<code>tsoutliers(gold_price\$USD)</code>	<pre>\$index integer(0) \$replacements numeric(0)</pre>
<code>tsoutliers(us_inflation\$Inflation_Rate)</code>	<pre>\$index integer(0) \$replacements numeric(0)</pre>
<code>tsoutliers(usd_index\$USD_Index)</code>	<pre>\$index integer(0) \$replacements numeric(0)</pre>
<code>tsoutliers(interest_rates\$interest_rates)</code>	<pre>\$index integer(0) \$replacements numeric(0)</pre>

Figure 1.8: Further data cleaning for individual datasets

Description: Another crucial part is checking for any outlier values and none of the above variables contain any outlier values.

Part C: Data Merging

```
data <- list(
  golddemand_us,
  gold_price,
  us_inflation,
  usd_index,
  interest_rates
) %>%
  reduce(full_join, by = "Quarter") %>%
  arrange(Quarter)
```

Figure 1.9: Code for merging all the datasets

Description: After cleaning individual datasets and prepping it, the above code shows the merging of each individual dataset into one full dataset named “data”. And it merged together based on Quarter from each dataset.

```
> sum(is.na(data))  
[1] 10
```

Figure 1.10: Checking for NA values after merging

Description: Once the data have been merged, another thorough check of any missing values that could be present and there are 10.

```
data_clean <- data %>%  
  arrange(Quarter) %>%  
  mutate(across(  
    where(is.numeric),  
    ~na.interp(ts(.x, frequency = 4))  
  ))
```

Figure 1.11: Cleaning full merged dataset

Description: Therefore, to ensure a proper clean dataset with no missing values, the number of missing values is 0.

```
golddemand_us <- ts(golddemand_us$Gold_Demand_Tonnes, start = c(2010,1), frequency = 4)  
gold_pricets <- ts(gold_price$USD, start = c(2010, 1), frequency = 4)  
us_inflationts <- ts(us_inflation$Inflation_Rate, start = c(2010,1), frequency = 4)  
interest_ratets <- ts(interest_rates$interest_rate, start = c(2010,1), frequency = 4)  
data_ts <- ts(data_clean, start = c(2010, 1), frequency = 4)
```

Figure 1.12: Converting into time series

Part D: Checking stationarity

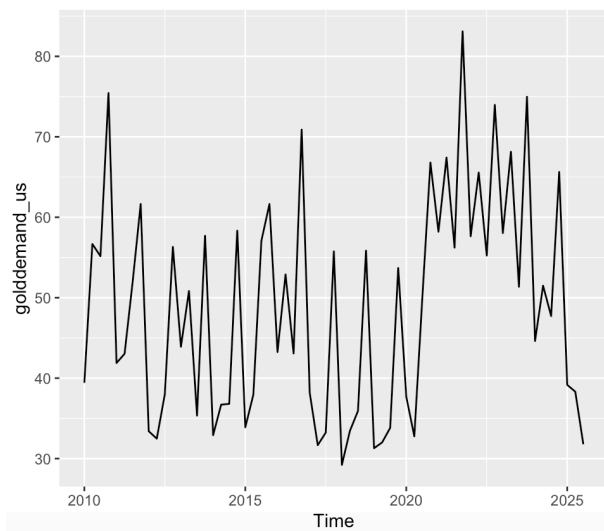


Figure 1.13: Autoplot plot for gold demand

Description: The quarterly demand for gold does not show any upward or downward trend, except for a sharp increase in the year of 2020. The variance looks relatively stable, however the mean does not relatively look stable.

<code>lambda_gdus <- BoxCox.lambda(golddemand_us)</code>	1.3149
<code>ndiffs(golddemand_us)</code>	0
<code>nsdiffs(golddemand_us)</code>	1
<code>adf.test(golddemand_us)</code>	Augmented Dickey-Fuller Test data: golddemand_us Dickey-Fuller = -1.6185, Lag order = 3, p-value = 0.7294 alternative hypothesis: stationary

Description: From checking the lambda value, it gives an output value of 1.3149 which suggests that a transformation may not be necessary as the lambda value is greater than 1. In addition, checking for any necessary first differencing with `ndiffs()` suggests that first differencing is not needed. However, one seasonal difference would be needed. With the ADF test, the p-value shows 0.7294 which is much higher than 0.05 which means it fails to reject the null hypothesis and it is non-stationary. Therefore, first differencing will be applied.

```
golddemand_us %>% diff() %>% diff(lag = 4) %>% ggtsdisplay()
```

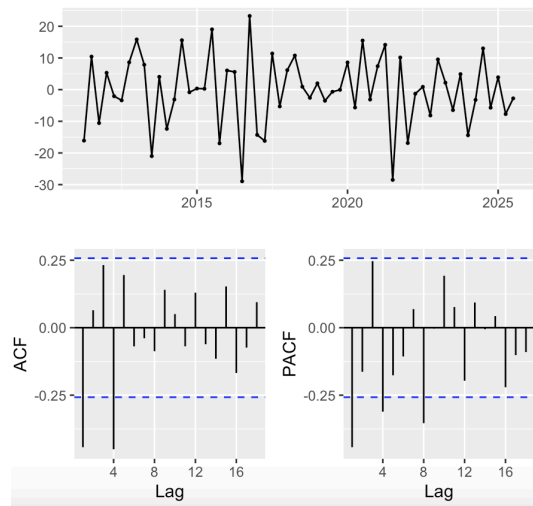


Figure 1.14: ggtsdisplay() after first differencing and seasonal differencing

Description: After applying first differencing, the mean is relatively stable, the ACF plot has no autocorrelation.

```
> adf.test(diff(golddemand_us))
```

Augmented Dickey-Fuller Test

```
data: diff(golddemand_us)
```

```
Dickey-Fuller = -4.0502, Lag order = 3, p-value = 0.01332
```

```
alternative hypothesis: stationary
```

Figure 1.15: Run ADF test after transformation

Description: After the first differencing is applied using the ADF test again to ensure that the p-value is smaller than 0.05 and the output is 0.0133, which is smaller than 0.05 which would suggest that it is now stationary.

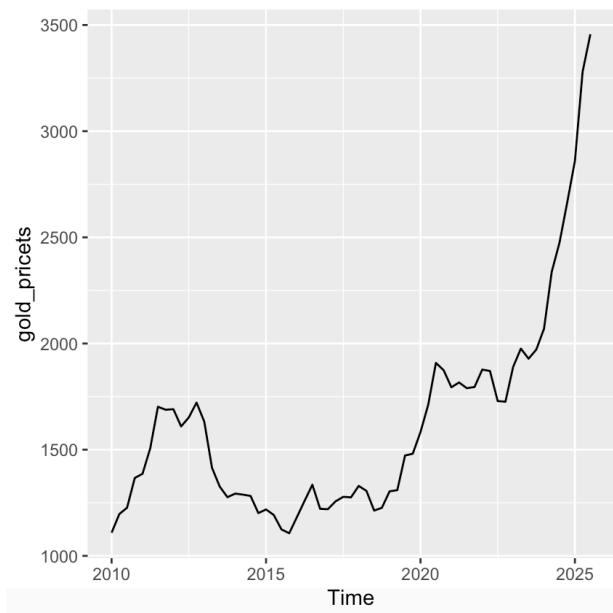


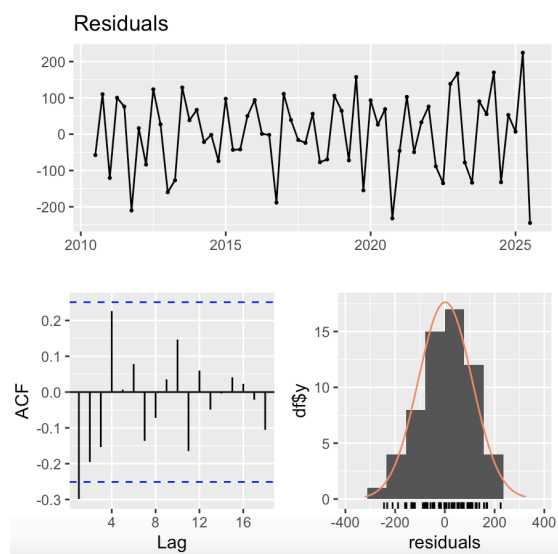
Figure 1.16: Autoplot of gold prices

Description: The plot for gold prices shows a significant increase towards the end, starting from 2024. The variance is relatively stable, however, the mean is non-constant.

<code>lambda_gp <- BoxCox.lambda(gold_pricets)</code>	-0.5968
<code>ndiffs(gold_pricets)</code>	2
<code>nsdiffs(gold_pricets)</code>	0
<code>adf.test(gold_pricets)</code>	Augmented Dickey-Fuller Test data: gold_pricets Dickey-Fuller = 0.57346, Lag order = 3, p-value = 0.99 alternative hypothesis: stationary

Description: By checking the lambda value, the need of transformation can be determined. Considering the lambda value is -0.5968 and that variance is relatively stable, the need of a log transformation may not be necessary. However, from the `ndiffs()` output, 2 first differencing is needed to stabilize the mean and none needed for seasonal differencing. From the ADF test, the p-value is significant and therefore first differencing would be applied.

```
gold_pricets %>% diff() %>% diff() %>% checkresiduals()
```



```
> adf.test(diff(diff(gold_pricets)))
```

Augmented Dickey-Fuller Test

```
data: diff(diff(gold_pricets))
Dickey-Fuller = -6.1435, Lag order = 3, p-value = 0.01
alternative hypothesis: stationary
```

Figure 1.17: Statistics check for gold prices

Description: Two first differencing is applied as with first differencing, the p-value is still larger than 0.05. However after 2 first differencing the p-value is smaller than 0.05, which suggests that the data is now stationary.

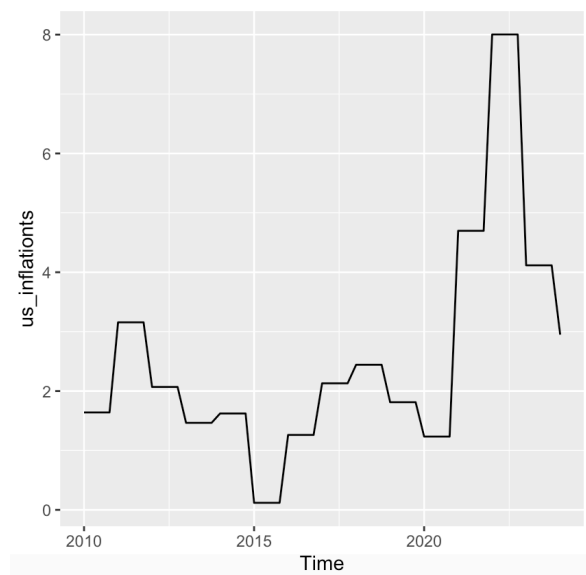


Figure 1.18: Autoplot plot for inflation

Description: The inflation rate shows that in the year of 2021, the inflation rate in the United States had significantly increased and continued to increase till 2023. The variance in the plot is somewhat stationary, however the mean is non-constant.

<code>lambda_if <- BoxCox.lambda(us_inflations)</code>	0.5730
<code>ndiffs(us_inflations)</code>	1
<code>nsdiffs(us_inflations)</code>	0
<code>adf.test(us_inflations)</code>	Augmented Dickey-Fuller Test data: us_inflations Dickey-Fuller = -2.37, Lag order = 3, p-value = 0.4254 alternative hypothesis: stationary

Description: By checking the lambda value, the use of either Boxcox transformation or log transformation would be needed. By checking whether a first differencing is needed and a seasonal differencing is needed, it shows one first differencing would be needed. However, using the ADF test for stationarity shows that the p-value is much greater than 0.05 which suggests a first difference is definitely needed.

```
us_inflations %>% diff() %>% diff() %>% ggtsdisplay()
adf.test(diff(diff(us_inflations)))
```

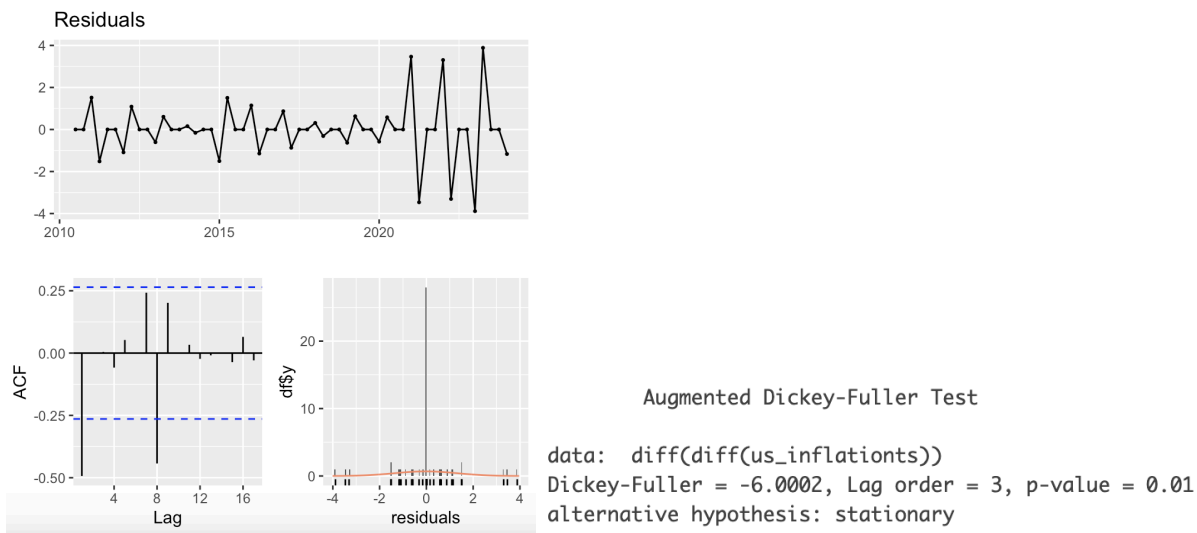


Figure 1.19: Statistics check for Inflation

Description: Two first differencing is needed to achieve a stationary data when the p-value is smaller than 0.05. From the ggtsdisplay() output, it also shows that there is somewhat a constant mean, no autocorrelation in the ACF plot.

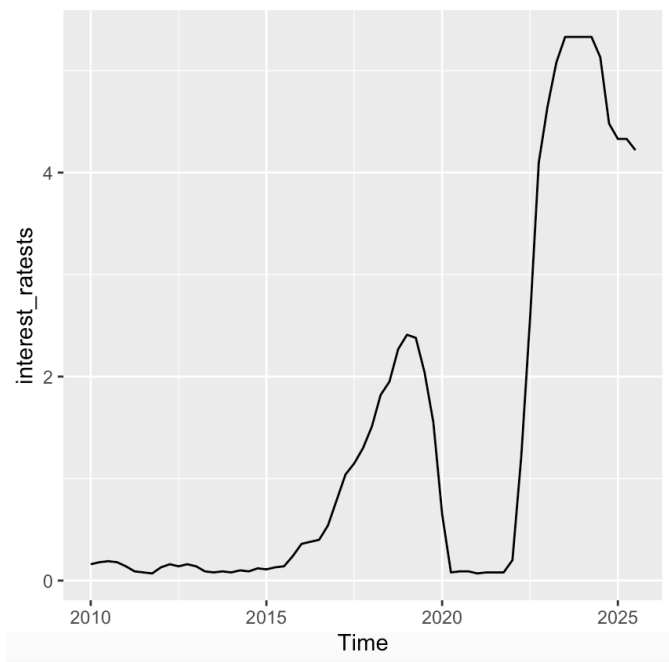


Figure 1.20: Autoplot plot for interest rate dataset

Description: The above plot for the interest rate shows that from the years 2010 to 2015, the interest rate seems to be stable. However, from 2017 onwards, there were significant fluctuations.

<code>lambda_ir <- BoxCox.lambda(interest_ratests)</code>	-0.6505
<code>ndiffs(interest_ratests)</code>	1
<code>nsdiffs(interest_ratests)</code>	0
<code>adf.test(interest_ratests)</code>	Augmented Dickey-Fuller Test data: interest_ratests Dickey-Fuller = -2.8152, Lag order = 3, p-value = 0.2448 alternative hypothesis: stationary

Description: The lambda value suggests that a boxcox or log transformation may not be needed. With the output of ndiffs() suggests that one differencing would be needed and none for

seasonal differencing. However, the ADF test shows a significant p-value which suggests that it fails to reject the null hypothesis and therefore the data is non-stationary and first differencing would be needed.

```
interest_ratests %>% diff() %>% diff() %>% checkresiduals()  
adf.test(diff(diff(interest_ratests)))
```

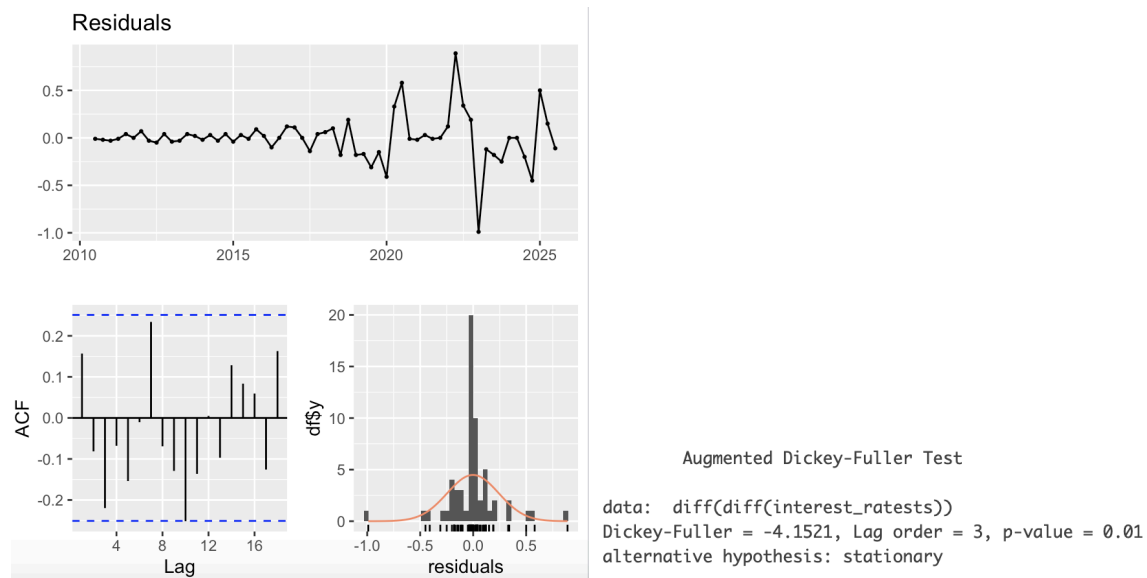


Figure 1.21: Statistics check for interest rates

Description: It appears that two first differencing is needed. As the p-value is now smaller than 0.05 which the data is now stationary.

- Are there seasonal patterns or structural breaks in the gold demand series?

```
decomp <- stl(golddemand_us, s.window = "periodic")
autoplot(decomp)
```

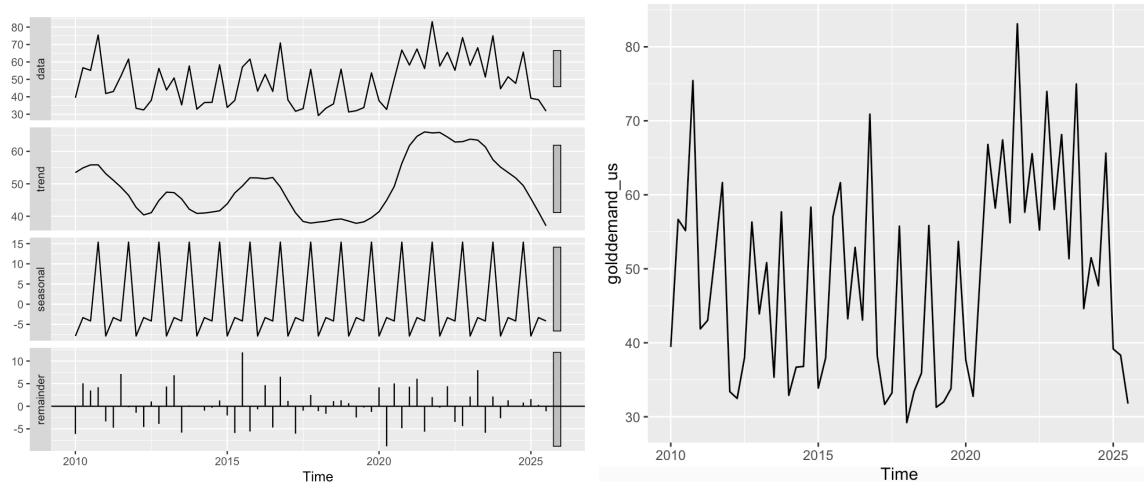


Figure 1.22: Seasonal decomposition and plot for golddemand_us and

Description: By using the `stl()` function, there is a clear seasonal pattern in the gold demand dataset. By visualisation on the plot for the gold demand in the United States, there is somewhat of a structural break in the year 2020 due to a sudden surge in the demand for gold.

- How did outliers affect the gold demand series? Did cleaning them change trends?

From the gold demand series, there were no outliers present. Therefore, there were no changes in the trend of the demand for gold.

Section 2: Regression with ARIMA Errors

Hypothesis Test:

H_0 : There is no significant relationship and influence between the dependent variable (gold demand) with the predictor variables (USD, inflation rate, USD index and interest rate).

H_1 : There is a significant relationship and influence between the predictor variable with the dependent variable.

```
fitmr <- tslm(
  Gold_Demand_Tonnes ~ USD + Inflation_Rate + USD_Index + interest_rate,
  data = data_ts
)
```

Figure 2.1: Fitting a linear regression model

Description: The above code illustrates the relationship between the dependent variable which is gold demand and the predictor variables.

```
> summary(fitmr)
```

Call:

```
tslm(formula = Gold_Demand_Tonnes ~ USD + Inflation_Rate + USD_Index +  
      interest_rate, data = data_ts)
```

Residuals:

Min	1Q	Median	3Q	Max
-18.649	-9.831	-1.443	8.390	30.847

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	5.896e+01	2.214e+01	2.663	0.00996 **
USD	-4.791e-04	4.019e-03	-0.119	0.90552
Inflation_Rate	3.192e+00	9.856e-01	3.238	0.00198 **
USD_Index	-1.384e+01	1.785e+01	-0.776	0.44108
interest_rate	-8.251e-01	1.353e+00	-0.610	0.54445

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 12.73 on 59 degrees of freedom

Multiple R-squared: 0.1782, Adjusted R-squared: 0.1225

F-statistic: 3.199 on 4 and 59 DF, p-value: 0.01911

Figure 2.2: Summary output for the linear regression model

Description: The predictor variable that has the strongest coefficient is the inflation rate. A 1% point increase in the inflation rate would result in a 3.19% increase in the change in demand for gold. This is expected as when inflation increases, it fuels demand for gold as the value of gold increases (bigseocontact, 2025). Overall the model is significant as the p-value of 0.01911 is smaller than 0.05, which also suggests that the null hypothesis is rejected.

```
> checkresiduals(fitmr)
```

Breusch-Godfrey test for serial correlation of order up to 8

data: Residuals from Linear regression model
LM test = 29.804, df = 8, p-value = 0.000229

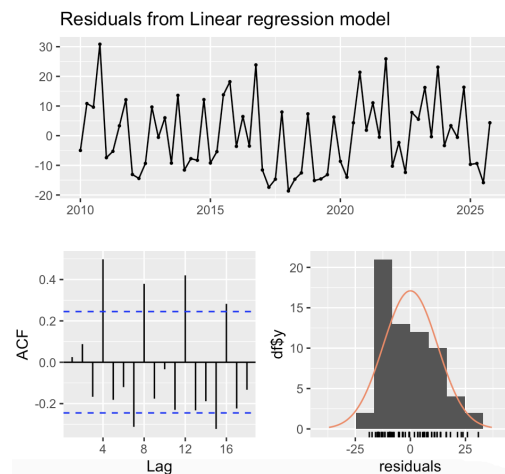


Figure 2.3: Checkresiduals() output for the linear regression model

Description: From the output, despite the p-value being smaller than 0.05, the checkresiduals() plot shows that it is not white noise as the mean is not constant, there is autocorrelation in the ACF plot and the residuals plot is relatively not normally distributed. Therefore, an ARIMA error model would be needed.

```
> coefficients(fitmr)
      (Intercept)      USD_Inflation_Rate      USD_Index interest_rate
      5.896189e+01 -4.791217e-04    3.191891e+00 -1.384205e+01 -8.250735e-01
```

Figure 2.4: Coefficient of the linear regression model

```
fit <- auto.arima(data_clean$Gold_Demand_Tonnes,
                  xreg = cbind(
                    data_clean$USD,
                    data_clean$Inflation_Rate,
                    data_clean$USD_Index,
                    data_clean$interest_rate
                  ))
```

Figure 2.5: ARIMA error model

Description: The above code illustrates the building of an ARIMA error model. The entire ARIMA error model is made into 'fit'. The dependent variable in this model would be the gold demand and the predictor variables are combined together under 'xreg='. As the function xreg= will contain the other variables that would help explain and predict the time series.

```
> summary(fit)
Series: data_clean$Gold_Demand_Tonnes
Regression with ARIMA(2,0,0)(2,1,0)[4] errors

Coefficients:
      ar1      ar2      sar1      sar2 data_clean$USD data_clean$Inflation_Rate
      0.4934  0.2689 -0.6167 -0.3621      -0.0048              0.6455
s.e.    0.1341  0.1345  0.1490  0.1438      0.0066              1.0751
      data_clean$USD_Index data_clean$interest_rate
              0.3694              -1.5720
s.e.              21.4828              1.6517

sigma^2 = 71.16; log likelihood = -209.88
AIC=437.76 AICc=441.36 BIC=456.61

Training set error measures:
              ME      RMSE      MAE      MPE      MAPE      MASE      ACF1
Training set -0.05792603 7.603635 5.560933 -1.670126 11.81093 0.6631516 -0.01034225
```

Figure 2.6: Summary statistics of the ARIMA error model

Description: The ARIMA error model suggests that the ARIMA model is ARIMA(2,0,0)(2,1,0).

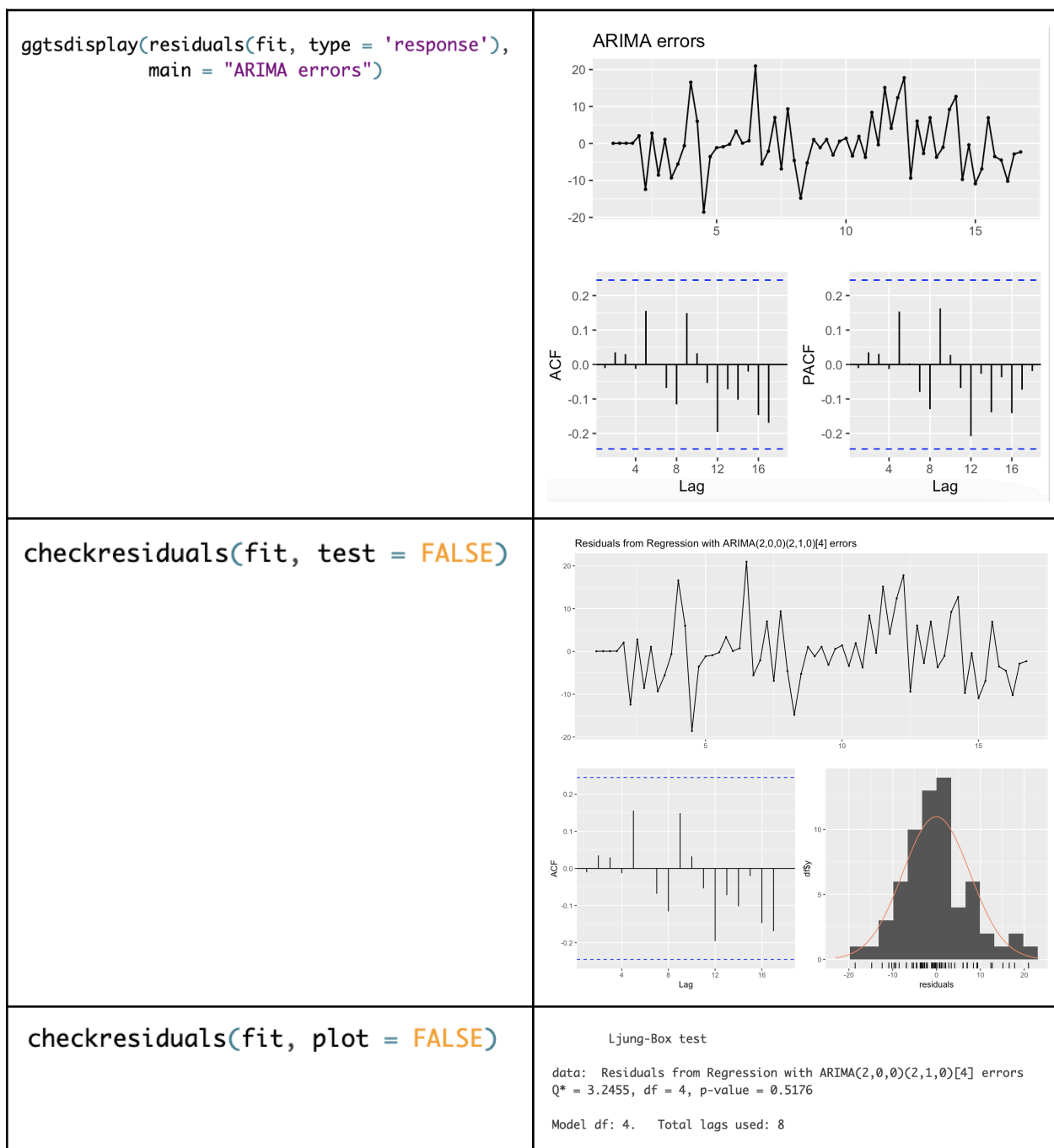


Figure 2.7: Checkresiduals() output for the ARIMA error model

```
fcast <- forecast(fit, xreg = cbind(
  USD = rep(mean(data_clean$USD), 12),
  Inflation_Rate = rep(mean(data_clean$Inflation_Rate), 12),
  USD_Index = rep(mean(data_clean$USD_Index), 12),
  interest_rate = rep(mean(data_clean$interest_rate), 12)
))

autoplot(fcast) +
  xlab("Year") +
  ylab("Change in gold demand (in Tonnes)") +
  ggtitle("Forecast with ARIMA(2,0,0)(2,1,1) errors")
```

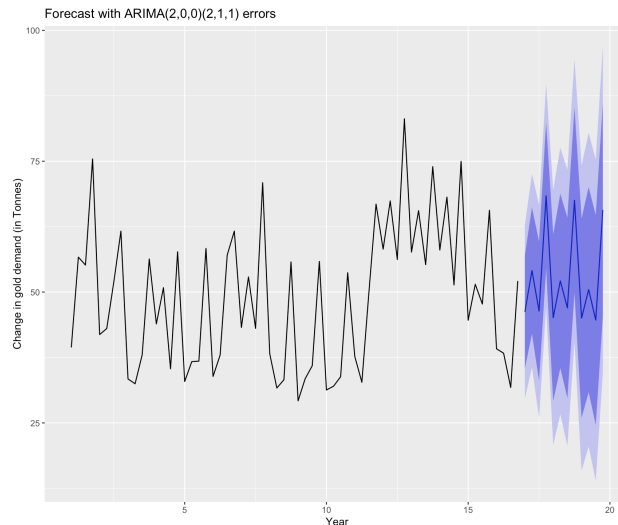


Figure 2.8: Forecast with the ARIMA error model

Description: The forecast with the ARIMA error model seems quite accurate for the next 12 quarters (3 years). Looking at the past historical data and pattern, the demand for gold seems to stay relatively stable for a period of time and either decreases or increases.

Section 3: Advanced Forecasting Method

Part A: Forecast Combination (Ensemble)

Hypothesis:

H_0 : The ensemble forecast did not achieve a low forecast error, determined from the comparison of the RMSE value.

H_1 : The ensemble forecast did achieve a low forecast error, determined from the comparison of RMSE values.

```
y <- data_ts[, "Gold_Demand_Tonnes"]
train_set <- window(y, end = c(2019, 4))
test_set <- window(y, start = c(2020, 1))
h <- length(test_set)

ETS <- forecast(ets(train_set), h = h)
ARIMA <- forecast(auto.arima(train_set, lambda = 0), h = h)
STLF <- stlf(train_set, lambda = 0, h = h)
Combination <- (ETS[["mean"]] + ARIMA[["mean"]] + STLF[["mean"]])/3
```

Figure 2.9: Ensemble forecast model

Description: The above code illustrates the building of the ensemble forecasts. The first lines of code shows defining the training and testing set with the interest of the gold demand variable.

The second chunk of codes shows the individual models, ETS, auto.arima and STL and lastly the combination model which takes the average of each individual model.

- Did the ensemble model perform better than individual models? Why might that be?

ETS	ARIMA	STLF	Combination
31.2321	32.1279	30.1068	31.0158

From the accuracy() test for the models, the STLF model performed better compared to other models, including the ensemble model. This could be due to the fact that the seasonal component in the gold demand is significant, as in the earlier stage, a seasonal pattern is present in the data. In addition to this, it makes the combination model weaker due to the other individual models like ETS and ARIMA to be much weaker, pulling the average of the combination model results to be lower.

- What are the benefits of combining forecasts in volatile markets like gold?

In volatile markets, such as gold, single models are unable to capture all the different aspects of the impact and factors that go into predicting and forecasting gold. According to Taneva-Angelova et al., (2025), an ensemble or combining forecast helps stabilize predictions when there is a significant amount of uncertainty and volatility and methods like these are more flexible and are easily adaptable to sudden changes.

```
autoplot(y) +  
  autolayer(ETS, series = "ETS", PI = FALSE) +  
  autolayer(ARIMA, series = "ARIMA", PI = FALSE) +  
  autolayer(STLF, series = "STL", PI = FALSE) +  
  autolayer(Combination, series = "Combination") +  
  xlab("Year") + ylab("Gold demand (in Tonnes)") +  
  ggtitle("Ensemble Forecast Comparison on Gold Demand")
```

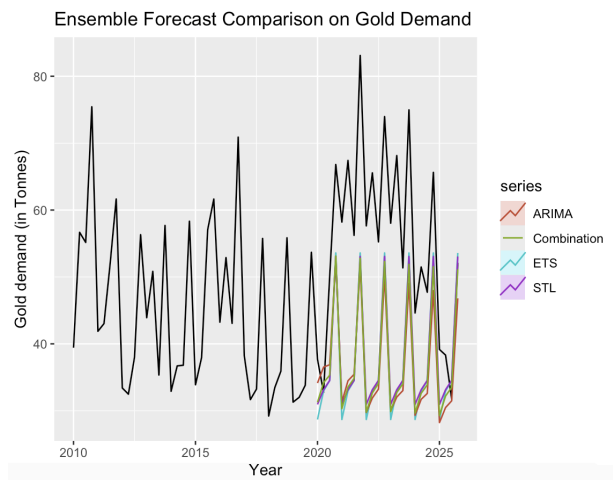


Figure 2.10: Visualisation of individual model with ensemble forecast

Description: From all the individual models and the combination forecast, it has predicted forecasts below the actual data. This shows the unpredictability and volatility of the gold market.

With models such as the individual models, it is built and relied on past data and patterns. Therefore, it would not have captured the sudden surge in the gold demand.

Part B: Bootstrapping and Bagging

Hypothesis:

H₀: The forecast model did not improve the forecast accuracy and did not accurately measure the forecast uncertainty.

H₁: The forecast model did improve the forecast accuracy and did accurately measure the forecast uncertainty.

```
baggedfc <- baggedModel(  
  y = train_set,  
  bootstrapped_series = bld.mbb.bootstrap(train_set, 20),  
  fn = ets  
) %>%  
  forecast(h = h)
```

Figure 2.11: Bootstrapping and bagging codes

Description: The above code illustrates generating the bagged forecasts from the bootstrapped series. To break down the code, the model will use the training set values. The following code `bootstrapped_series = bld.mbb.bootstrap(train_set, 20)` will create 20 new time series which is done by resampling with sampling replacements from an existing sample data. The code 'fn = ets' function is to fit an ETS model into each of the 20 time series. The `baggedModel()` will then produce the forecasts by aggregating it.

```
y <- data_ts[, "Gold_Demand_Tonnes"]  
autoplot(y) +  
  autolayer(baggedfc, series = "Bagging ETS", PI = F) +  
  xlab("Year") + ylab("Gold Demand (in Tonnes)") +  
  ggtitle("Comparison against actual data")
```

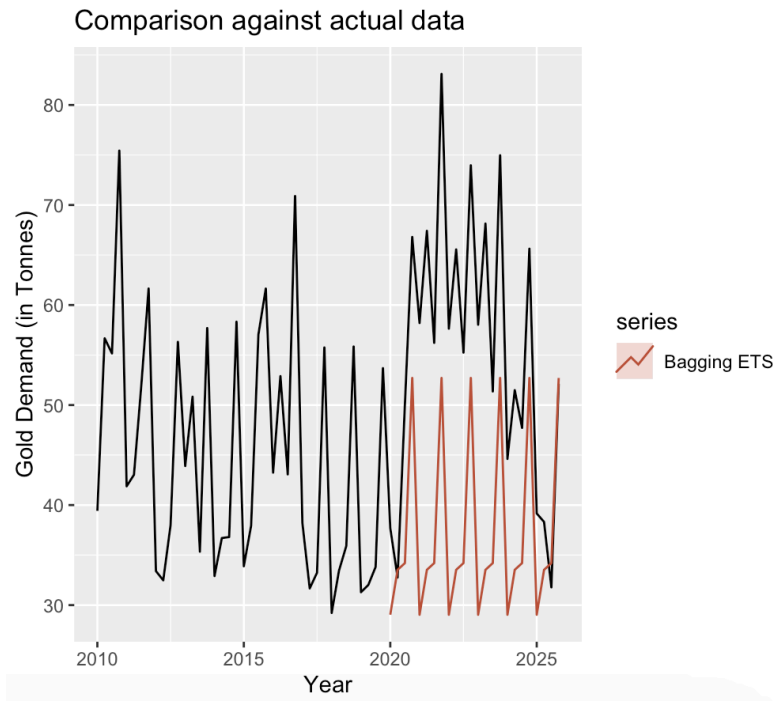


Figure 2.12: Forecasted bagging values against the actual data

Description: The above code plots the forecast from the BaggedModel() which uses the ETS forecasts prediction shows a constant similar trend from 2020 onwards compared to the actual gold demand data.

- How does bootstrapping help capture uncertainty in your time series?

How bootstrapping helps capture uncertainty is from resampling data from the existing data which then every resampled data would be fitted with a model, such as ETS as used above to generate a forecast. This would in turn quantify the amount of uncertainty in the forecasts as it not only fully relies on model assumptions from the pattern and structure of the past data but from the variability of the actual data (Petropoulos et al., 2018). However, with bootstrapping for this time series, it might not have captured the uncertainty as with Figure 2.12, the forecasted values remained constant and similar with the past data.

- Did bagging reduce your forecast error? In what types of series is bagging most useful?

	ME	RMSE	MAE	MPE	MAPE	MASE	ACF1	Theil's U
ETS	17.89300	20.96137	18.26044	30.22721	31.23209	2.369255	0.6343230	1.166881
ARIMA	18.62067	21.45750	18.93437	31.21036	32.16793	2.456696	0.6069444	1.213759
Bagged	17.09413	20.24476	17.59036	28.50300	29.94148	2.282312	0.6343713	1.132382

From the above table which shows the comparison between individual models with the bagged model, the bagged model did in fact reduce the forecast error as it has the lowest RMSE values. Bagging would be most useful for time series which are high in variance (IBM, 2021).

Section D: Model Evaluation & Selection

Part A: Model from Section B

```
> accuracy(fit)
```

	ME	RMSE	MAE	MPE	MAPE	MASE
Training set	-0.05792603	7.603635	5.560933	-1.670126	11.81093	0.6631516

```
ACF1
```

	ACF1
Training set	-0.01034225

Figure 4.1: Traditional values for the linear regression model

```
y <- data_ts[, "Gold_Demand_Tonnes"]
xreg <- cbind(
  data_clean$USD,
  data_clean$Inflation_Rate,
  data_clean$USD_Index,
  data_clean$interest_rate
)
h <- 12

e_autoarima <- tsCV(
  y = y,
  forecastfunction = function(y, h, xreg, newxreg) {
    fit <- auto.arima(y, xreg = xreg)
    forecast(fit, h = h, xreg = newxreg)},
  h = h, xreg = xreg)

rmse_arima <- sqrt(colMeans(e_arima^2, na.rm = TRUE))
rmse_arima
```

	h=1	h=2	h=3	h=4	h=5	h=6	h=7	h=8	h=9	
	10.82629	12.09644	13.28114	13.65356	14.64053	15.28877	15.97092	16.14624	16.41499	
	h=10	h=11	h=12							
	17.25917	17.38209	17.36140							

Figure 4.2: TSCV values for the linear regression model

Part B: Model from Section 3-Part A

```

> accuracy(ETS, test_set)
      ME      RMSE      MAE      MPE      MAPE      MASE
Training set -0.6026352  6.384028  4.682176 -3.028107 10.82837 0.6075025
Test set     17.8929951 20.961372 18.260445 30.227208 31.23209 2.3692545
      ACF1 Theil's U
Training set 0.02988615      NA
Test set     0.63432305  1.166881
> accuracy(ARIMA, test_set)
      ME      RMSE      MAE      MPE      MAPE      MASE
Training set 0.06406602  5.315363  4.023228 -1.536727  9.264631 0.5220054
Test set     18.62066634 21.457496 18.934375 31.210360 32.167933 2.4566956
      ACF1 Theil's U
Training set -0.07839172      NA
Test set     0.60694438  1.213759
> accuracy(STLF, test_set)
      ME      RMSE      MAE      MPE      MAPE      MASE
Training set -0.4692118  6.206686  4.575154 -2.431752 10.41538 0.5936166
Test set     17.4120450 20.488236 17.752010 29.134723 30.10638 2.3032861
      ACF1 Theil's U
Training set -0.1101592      NA
Test set     0.6306467  1.15299
> accuracy(Combination, test_set)
      ME      RMSE      MAE      MPE      MAPE      ACF1 Theil's U
Test set 17.97524 20.92912 18.24101 30.19076 31.01575 0.6276305 1.173956

```

Figure 4.3: Traditional values for the ensemble forecast model

Description: From the ensemble forecasts, the STLF has the lowest RMSE values which suggests that with comparison between the other models, it has a more accurate forecast prediction.

```

y <- data_ts[, "Gold_Demand_Tonnes"]
h <- 12

e_ets <- tsCV(y, forecastfunction = function(y, h) forecast(ets(y), h = h), h = h)
e_arima <- tsCV(y, forecastfunction = function(y, h) forecast(auto.arima(y, lambda = 0), h = h), h = h)
e_stlf <- tsCV(y, forecastfunction = function(y, h) stlf(y, lambda = 0, h = h), h = h)
e_ensemble <- (e_ets + e_arima + e_stlf)/3

rmse_ensemble <- sqrt(colMeans(e_ensemble^2, na.rm = TRUE))
rmse_ensemble

      e_ets + e_arima.e_ets.h=1  e_ets + e_arima.e_ets.h=2  e_ets + e_arima.e_ets.h=3
                        8.257148                        9.425247                        10.239701
      e_ets + e_arima.e_ets.h=4  e_ets + e_arima.e_ets.h=5  e_ets + e_arima.e_ets.h=6
                        11.589246                        12.689022                        13.780379
      e_ets + e_arima.e_ets.h=7  e_ets + e_arima.e_ets.h=8  e_ets + e_arima.e_ets.h=9
                        14.198967                        15.089622                        15.654152
      e_ets + e_arima.e_ets.h=10 e_ets + e_arima.e_ets.h=11 e_ets + e_arima.e_ets.h=12
                        16.568381                        16.522253                        17.017519

```

Figure 4.4: TSCV values for the ensemble method/model

Part C: Model from Section 3-Part B

```
> accuracy(baggedfc, test_set)
```

	ME	RMSE	MAE	MPE	MAPE	MASE	ACF1	Theil's U
Training set	-0.670013	5.503009	4.160373	-3.059834	9.693489	0.5397998	0.2187388	NA
Test set	17.594458	20.631185	17.960432	29.491515	30.567066	2.3303285	0.6456798	1.154988

Figure 4.5: Traditional values for bagging model.

```
bagged_forecast_fn <- function(y, h) {
  fit <- baggedETS(y, B = 20)
  fc <- forecast(fit, h = h)
  return(as.numeric(fc))}

e_bagged <- tsCV(
  y = train_set,
  forecastfunction = bagged_forecast_fn,
  h = 12)
rmse_bagged <- sqrt(colMeans(e_bagged^2, na.rm = TRUE))
rmse_bagged
```

h=1	h=2	h=3	h=4	h=5	h=6	h=7	h=8	h=9	h=10	h=11	h=12
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

Figure 4.6: TSCV values for bagging model.

Comparison between all the models

Out-of-sample forecast error values:

	RMSE	MAE	MAPE
ARIMA error model	7.60	5.5609	11.8109
Ensemble model			
ETS	20.9614	18.2604	31.2321
ARIMA	21.4575	18.9344	32.1679
STLF	20.4882	17.7520	30.1064
Combination	20.9291	18.2410	31.0158
Bagging ETS model	20.2448	17.5904	29.9415

Out of the three models, the ARIMA error model has the lowest out-of-sample forecast error. However, this is due to the fact that the ARIMA error model is built on the entire dataset, whereas the ensemble forecast model and the bootstrap and bagging model is built on a

training and testing set. Therefore, it would be expected that the ARIMA error model to have a much lower out-of-sample forecast error as it also includes the future values, which for the other two models were the test_set.

Cross validation values comparison between all the models:

	ARIMA error model	Ensemble model	Bagged ETS model
h=1	10.82629	8.257148	NaN
h=2	12.09644	9.425247	NaN
h=3	13.28114	10.239701	NaN
h=4	13.65356	11.589246	NaN
h=5	14.64053	12.689022	NaN
h=6	15.28877	13.780379	NaN
h=7	15.97092	14.198967	NaN
h=8	16.14624	15.089622	NaN
h=9	16.41499	15.654152	NaN
h=10	17.25917	16.568381	NaN
h=11	17.38209	16.522253	NaN
h=12	17.36140	17.017519	NaN

The above cross validation values show NaN for the bagged ETS model, which could be due to the length of the test_set being too short for the forecast horizon. However in this situation, the ensemble model would be the best performing model as it has the lowest cross validation values.

Section E: Interpretation and Reflection

The model built in Section B, the linear regression model and ARIMA error forecast model, helped determine which of the predictor variables had the most influence on the demand for gold. In addition to this, the ARIMA error forecast model incorporates the external predictors, such as USD, interest rates, inflation rate and the USD index rather than just with one variable. The downside of this is that with the ARIMA error model, all the variables need to be stationary

with the same differencing applied to all of them. If there are variables which are still non-stationary, it causes the estimated coefficients to be incorrect.

For the model built in Section C part A, ensemble forecast, the model's advantage is normally known to improve the predictive performance by aggregation. However, in the models performed above, the individual STLF model outperformed the combination model. Therefore, it shows that although ensemble models are highly recommended for high unpredictability and volatile data, there are some cases like this where the gold demand series could be much more stable and predictable, and as a result the STLF model would be the best performing model.

For the model built in Section C part B, bootstrapping and bagging can be tedious due to ordering and autocorrelation in a time series data. However, the model itself does significantly improve in predicting its forecast as its RMSE values as printed above is the lowest among other models.

References

- bigseocontact. (2025, May 2). *Gold, Inflation and Interest Rates: How Do They Interact?*
FocusEconomics.
<https://www.focus-economics.com/blog/gold-inflation-interest-rates-prices-impact/>
- Board of Governors of the Federal Reserve System (US). (2025). *U.S. / Euro Foreign Exchange Rate*. FRED, Federal Reserve Bank of St. Louis.
<https://fred.stlouisfed.org/series/DEXUSEU>
- Federal Reserve Bank of St. Louis. (2025). *Effective Federal Funds Rate*. Stlouisfed.org.
<https://fred.stlouisfed.org/series/FEDFUNDS>
- IBM. (2021, September 23). *Bagging*. Ibm.com. <https://www.ibm.com/think/topics/bagging>
- Petropoulos, F., Hyndman, R. J., & Bergmeir, C. (2018). Exploring the sources of uncertainty: Why does bagging for time series forecasting work? *European Journal of Operational Research*, 268(2), 545–554.
<https://doi.org/10.1016/j.ejor.2018.01.045>
- Taneva-Angelova, G., Raychev, S., & Ilieva, G. (2025). A Framework for Gold Price Prediction Combining Classical and Intelligent Methods with Financial, Economic, and Sentiment Data Fusion. *International Journal of Financial Studies*, 13(2), 102.
<https://doi.org/10.3390/ijfs13020102>
- World Gold Council. (2025). *Gold Demand by Country | Gold Demand and Supply Statistics | Goldhub*. World Gold Council.
<https://www.gold.org/goldhub/data/gold-demand-by-country>

World Gold Council. (2019, August 27). *Gold Price Historical Data | Gold Price History | World Gold Council*. World Gold Council; World Gold Council.
<https://www.gold.org/goldhub/data/gold-prices>

World Bank. (2024). *Inflation, consumer prices (annual %)*. Worldbank.org.
<https://data.worldbank.org/indicator/FP.CPI.TOTL.ZG>